

PRACTICAL WEEK-10

1 . Investigate inode structures using commands such as `ls -i`, `stat`, and `find`. Create hard links and symbolic links and explain their impact on inode allocation.

Aim : To study inode structures in Linux and understand the difference between hard links and symbolic links.

Program :

```
GNU nano 7.2  
This is the original file for Practical 10.
```

Output :



root@keerthana: ~/OSSP



```
root@keerthana:~# cd ~/OSSP
root@keerthana:~/OSSP# mkdir -p ~/OSSP
cd ~/OSSP
root@keerthana:~/OSSP# nano original.txt
root@keerthana:~/OSSP# ls -li original.txt
45708 original.txt
root@keerthana:~/OSSP# ls -li original.txt
45708 -rw-r--r-- 1 root root 44 Sep 28 02:54 original.txt
root@keerthana:~/OSSP# stat original.txt
  File: original.txt
  Size: 44          Blocks: 8          IO Block: 4096   regular file
Device: 8,48   Inode: 45708      Links: 1
Access: (0644/-rw-r--r--)  Uid: (   0/   root)   Gid: (   0/   root)
Access: 2026-09-28 02:54:43.501643218 +0000
Modify: 2026-09-28 02:54:43.503197681 +0000
Change: 2026-09-28 02:54:43.503197681 +0000
 Birth: 2026-09-28 02:54:43.501643218 +0000
root@keerthana:~/OSSP# find . -inum 45708
./original.txt
root@keerthana:~/OSSP# ln original.txt hardlink.txt
root@keerthana:~/OSSP# ls -li original.txt hardlink.txt
45708 -rw-r--r-- 2 root root 44 Sep 28 02:54 hardlink.txt
45708 -rw-r--r-- 2 root root 44 Sep 28 02:54 original.txt
root@keerthana:~/OSSP# stat original.txt
  File: original.txt
  Size: 44          Blocks: 8          IO Block: 4096   regular file
Device: 8,48   Inode: 45708      Links: 2
Access: (0644/-rw-r--r--)  Uid: (   0/   root)   Gid: (   0/   root)
Access: 2026-09-28 02:54:43.501643218 +0000
Modify: 2026-09-28 02:54:43.503197681 +0000
Change: 2026-09-28 02:56:07.579037305 +0000
 Birth: 2026-09-28 02:54:43.501643218 +0000
root@keerthana:~/OSSP# ln -s original.txt symlink.txt
root@keerthana:~/OSSP# ls -li hardlink.txt original.txt symlink.txt
45708 -rw-r--r-- 2 root root 44 Sep 28 02:54 hardlink.txt
45708 -rw-r--r-- 2 root root 44 Sep 28 02:54 original.txt
12891 lrwxrwxrwx 1 root root 12 Sep 28 02:56 symlink.txt -> original.txt
root@keerthana:~/OSSP# stat symlink.txt
  File: symlink.txt -> original.txt
  Size: 12          Blocks: 0          IO Block: 4096   symbolic link
Device: 8,48   Inode: 12891      Links: 1
Access: (0777/lrwxrwxrwx)  Uid: (   0/   root)   Gid: (   0/   root)
Access: 2026-09-28 02:56:46.380865065 +0000
Modify: 2026-09-28 02:56:39.813809460 +0000
Change: 2026-09-28 02:56:39.813809460 +0000
 Birth: 2026-09-28 02:56:39.813809460 +0000
root@keerthana:~/OSSP# ls -l symlink.txt
lrwxrwxrwx 1 root root 12 Sep 28 02:56 symlink.txt -> original.txt
root@keerthana:~/OSSP# find . -inum 45708
./hardlink.txt
./original.txt
root@keerthana:~/OSSP# rm -f original.txt hardlink.txt symlink.txt
root@keerthana:~/OSSP# ls
LICENSE
Makefile
output.txt
README.md
scripts
Week5
source.txt
Week6
src
Week7
task1
Week8
task1.c
assets
task_error_handling
bin
task_error_handling.c
calculator.c
task_process_groups
calculator.h
task_process_groups.c
calculator.o
task_resume_job
calculator_test
task_resume_job.c
calculator_test.c
task_sigint_forward
docs
task_sigint_forward.c
error.log
task_signal_actions
expected_report.txt
task_signal_actions.c
include
task_signal_behavior
input.txt
task_signal_behavior.c
io_redirect
task_sigsttp
io_redirect.c
task_sigsttp.c
library_copy
task_valgrind
library_copy.c
task_valgrind.c
library_copy.txt
task_valgrind_leak
lowlevel_copy
task_valgrind_leak.c
```

2 . Design a program using `mmap()` to perform file reading and writing. Compare memory-mapped I/O with traditional `read()/write()` operations in terms of efficiency and implementation complexity.

Aim : To implement file reading and writing using `mmap()` and compare it with traditional `read()/write()` operations.

Program :

```
GNU nano 7.2
Hello from mmap.
This is Practical 10.
Linux memory mapped file I/O.|
```

GNU nano 7.2

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <string.h>

int main()
{
    int fd;
    struct stat st;
    char *data;

    fd = open("mmap_file.txt", O_RDWR);

    if (fd == -1)
    {
        perror("open");
        return 1;
    }

    if (fstat(fd, &st) == -1)
    {
        perror("fstat");
        close(fd);
        return 1;
    }

    data = mmap(NULL, st.st_size,
                PROT_READ | PROT_WRITE,
                MAP_SHARED, fd, 0);

    if (data == MAP_FAILED)
    {
        perror("mmap");
        close(fd);
        return 1;
    }

    printf("Original content:\n");
    write(STDOUT_FILENO, data, st.st_size);

    memcpy(data, "Modified using mmap().\n", 23);

    if (msync(data, st.st_size, MS_SYNC) == -1)
    {
        perror("msync");
    }

    printf("\nFile modified successfully using mmap().\n");

    munmap(data, st.st_size);
    close(fd);

    return 0;
}
```

GNU nano 7.2

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

#define BUFFER_SIZE 1024

int main()
{
    int fd;
    ssize_t bytes_read;
    char buffer[BUFFER_SIZE];

    fd = open("mmap_file.txt", O_RDONLY);

    if (fd == -1)
    {
        perror("open");
        return 1;
    }

    bytes_read = read(fd, buffer, BUFFER_SIZE - 1);

    if (bytes_read == -1)
    {
        perror("read");
        close(fd);
        return 1;
    }

    buffer[bytes_read] = '\0';

    printf("Content using read():\n");
    printf("%s", buffer);

    close(fd);

    return 0;
}
```

Output :

```
root@keerthana: ~/OSSP
root@keerthana:~/OSSP# cd ~/OSSP
root@keerthana:~/OSSP# nano mmap_file.txt
root@keerthana:~/OSSP# ^C
root@keerthana:~/OSSP# cat mmap_file.txt
Hello from mmap.
This is Practical 10.
Linux memory mapped file I/O.
root@keerthana:~/OSSP# nano mmap_file.c
root@keerthana:~/OSSP# gcc -Wall -Wextra mmap_file.c -o mmap_file
root@keerthana:~/OSSP# ./mmap_file
Original content:
Hello from mmap.
This is Practical 10.
Linux memory mapped file I/O.

File modified successfully using mmap().
root@keerthana:~/OSSP# cat mmap_file.txt
Modified using mmap().
s Practical 10.
Linux memory mapped file I/O.
root@keerthana:~/OSSP# nano traditional_io.c
root@keerthana:~/OSSP# gcc -Wall -Wextra traditional_io.c -o traditional_io
root@keerthana:~/OSSP# ./traditional_io
Content using read():
Modified using mmap().
s Practical 10.
Linux memory mapped file I/O.

File modified successfully using mmap().

real    0m0.010s
user    0m0.001s
sys     0m0.000s
root@keerthana:~/OSSP# time ./traditional_io
Content using read():
Modified using mmap().
s Practical 10.
Linux memory mapped file I/O.

real    0m0.002s
user    0m0.000s
sys     0m0.000s
root@keerthana:~/OSSP# |
```